

// JavaJub

# Разбор задачи

## X5 Tech · Senior

4 из 8 багов в сервисе расчёта скидок

*Чёрная пятница в Пятёрочке · код-ревью под Senior*

### Что в этом документе

4 разбора с правильным кодом · тизеры на остальные 4 · главный коварный — в Pro

*Бесплатная версия из серии Java Jub Pro · Полный разбор + код + чек-лист в Pro*

# Содержание

|                                                                                   |                                                           |    |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------|----|
| 01                                                                                | Постановка задачи и контекст                              | 3  |
| 02                                                                                | Баг #1 — contention на shared Random (JMH-цифры внутри)   | 5  |
| 03                                                                                | Баг #2 — parallelStream на общем ForkJoinPool             | 8  |
| 04                                                                                | Баг #3 — magic filter в стриме                            | 11 |
| 05                                                                                | Баг #4 — field injection и не-final зависимости           | 13 |
|  | <i>Ещё 4 бага в Java Jub Pro (главный коварный — там)</i> | 15 |
|  | <i>Финальный production-ready код + чек-лист</i>          | 18 |

# 01 Постановка задачи и контекст

Эту задачу дают на Senior-собеседовании в X5 Tech и Farzoom. Снаружи выглядит как обычное «найди баги в коде на `parallelStream`». Внутри — самая многослойная задача из X5-серии: 8 разных багов в 15 строках, причём главный — не там, где обычно ищут. Если ты Senior и пропустишь главный — это сигнал интервьюеру, что в проде у тебя были бы инциденты на деньги.

В этой бесплатной версии разбираем 4 бага из 8 — техническая база, на которой отсеивают Middle. Оставшиеся 4 — самые провокационные, на них валятся даже Senior, и среди них тот САМЫЙ главный коварный, ради которого вся задача и существует на собесе. Полный разбор каждого с примерами правильного кода — в Java Jub Pro.

## ► Контекст для интервью

- Компания: X5 Tech / Farzoom, Senior Java.
- Бренд внутри X5: Пятёрочка, расчёт скидок для Чёрной пятницы.
- Стек: Java 21, Spring Boot 3, PostgreSQL.
- Масштаб: 50 000 SKU, цены пересчитываются перед открытием акции.
- Тайминг на собесе: 20 минут на разбор.
- Сложность: medium-hard.

## ► Исходный код

Ниже сервис, который прислал автор. Задача — найти баги. По уровню Senior их должно быть 6+, идеально 8. Прочти код медленно. На первый взгляд он выглядит «нормально», компилируется и даже отработывает. Запомни это ощущение — оно главный враг на этой задаче.

```
@Service
public class BlackFridayPricingService {

    private Random random = new Random();

    public List<BigDecimal> calculateDiscountPrices(int[] basePrices) {

        return Arrays.stream(basePrices)
            .parallel()
            .filter(price -> price % 2 != 0)
            .mapToObj(price -> {
                try {
                    Thread.sleep(50);
                } catch (InterruptedException e) {
                    // ignore
                }
                double discount = random.nextDouble() * 0.5;
                return new BigDecimal(price * (1 - discount));
            })
            .collect(Collectors.toList());
    }
}
```

## ► Стратегия разбора на собесе

1. Прочитай код 2 раза молча, без комментариев. Первый раз — общее понимание. Второй — примечай узкие места.

2. Начни вслух с типов данных (BigDecimal, double, int). Это покажет интервьюеру, что ты думаешь о финансовой корректности до многопоточки.
3. Дальше — многопоточный блок. Здесь Senior должен идти не «парадного» путём (parallel плох), а глубоким: ForkJoinPool, contention, escape carrier (Java 21).
4. Потом Random — и его две проблемы (contention + детерминизм).
5. В конце — стиль и архитектура: filter, final-поля, DI.

**ФИШКА** · Что показывает Senior на этой задаче

Задаёт уточняющие вопросы перед тем, как чертить — это первый признак опыта.

Отделяет «нашёл баг» от «обосновал что будет в проде» — второе ценится выше.

Начинает с самого опасного бага, не с самого очевидного.

Знает Goetz «Java Concurrency in Practice» — это видно по тому, как он говорит про InterruptedException.

## 02 Баг #1 — contention на shared Random

Начнём с самого контринтуитивного бага в коде. Технически `java.util.Random` thread-safe — это написано в Javadoc. На собесе кандидат говорит «ну, `Random` безопасен, я проверил». Дальше идут JMH-бенчмарки, которые показывают: `parallelStream` с shared `Random` может работать МЕДЛЕННЕЕ, чем тот же код в `sequential`-режиме. Это не баг производительности, это баг ПОНИМАНИЯ thread-safety.

### ► Где баг

```
private Random random = new Random();           // <-- один на сервис

// внутри parallelStream:
double discount = random.nextDouble() * 0.5;    // <-- 7 потоков дёргают одну ссылку
```

### ► JMH-цифры — увидеть глазами

Реальный замер на 8-ядерной машине, 1 миллион вызовов `nextDouble`:

| Подход                                              | Время  | Замечание                    |
|-----------------------------------------------------|--------|------------------------------|
| sequential (без parallel)                           | ~12 ms | baseline                     |
| parallel + shared <code>java.util.Random</code>     | ~25 ms | × медленнее sequential!      |
| parallel + <code>ThreadLocalRandom.current()</code> | ~2 ms  | × в 6 раз быстрее sequential |

Эти цифры контринтуитивны. Если ты раньше думал, что `.parallel()` ВСЕГДА ускоряет код — это контрпример. И он не теоретический. Любой может проверить через `jmh` за 10 минут. На собесе X5 спросят именно это: «почему `parallel` на shared `Random` медленнее, чем `sequential`?». Если ответа нет — Senior-оффер не светит.

### ► Что не так — внутреннее устройство `Random`

#### ■ Как `java.util.Random` становится thread-safe?

→ `java.util.Random` формально thread-safe — это написано в Javadoc. Но реализация потокобезопасности — через атомарный CAS на одном поле `AtomicLong seed`. Каждый вызов `nextDouble()` внутри делает Compare-And-Set операцию на этом `seed`. Когда 7 потоков параллельно долбят один `Random` — все 7 бьются за один и тот же CAS на одной переменной. Это называется contention (конкуренция за ресурс). CPU тратит больше времени на координацию между потоками, чем на полезную работу.

#### ■ Почему shared `Random` может быть медленнее `sequential`?

→ В `sequential` один поток последовательно дёргает `nextDouble` — CAS всегда успешен, никаких retry. В `parallel` 7 потоков одновременно пытаются обновить `seed`. Только один CAS успешен, остальные 6 потоков retry-ят. На следующей итерации картина повторяется: один успешен, шесть retry. CPU занят координацией, прогресса мало. Плюс — сами CPU-кэши проседают, потому что переменная `seed` постоянно инвалидируется через MESI-протокол между ядрами. Итог: `parallel` хуже `sequential` на 30-100%, в зависимости от железа.

### ► Решение: `ThreadLocalRandom`

```
double discount = ThreadLocalRandom.current().nextDouble() * 0.5;
```

#### ▪ Как устроен ThreadLocalRandom?

→ ThreadLocalRandom использует отдельный seed для каждого потока — хранится в полях класса Thread (это специальная JVM-оптимизация, не обычный ThreadLocal). Метод current() возвращает экземпляр для текущего потока без synchronized и без CAS — просто читает из локального состояния потока. Поэтому нет contention, каждый поток работает со своим seed независимо. Создан специально для concurrent кода. Появился в Java 7 (java.util.concurrent.ThreadLocalRandom). С Java 17 появился общий интерфейс RandomGenerator — рекомендуемый способ работать с ГСЧ в новом коде.

#### ▪ Когда использовать что — Random, SecureRandom, ThreadLocalRandom?

→ Random — для однопоточного кода с обычной случайностью (тесты, моки, простой код). ThreadLocalRandom — для многопоточного кода с обычной случайностью (concurrent workloads, parallel stream). SecureRandom — для криптографически стойких чисел: токены аутентификации, ID транзакций, salt для паролей. SecureRandom медленнее (использует энтропию ОС через /dev/urandom), но непредсказуемо. Никогда не используй Random для session ID или паролей — последовательность вычисляется по seed.

### ► Что должен видеть Senior на code review

- private Random random = new Random() как поле сервиса — автоматический маркер для проверки: «используется ли в parallel-контексте?»
- Если в коде есть .parallel() / parallelStream() рядом с shared Random — это блокирующее замечание на review.
- Правильно: ThreadLocalRandom.current() внутри лямбды или RandomGenerator через DI.
- В JMH-бенчмарке любого concurrent-кода — обязательно проверять с разными RNG, иначе микро-бенчмарк не показывает реальную картину прода.

#### ФИШКА • Что сказать на собесе про этот баг

«java.util.Random thread-safe формально, но через CAS на одном seed».

«В parallelStream это даёт contention — параллельный код медленнее sequential».

«Решение — ThreadLocalRandom.current() (Java 7+) или RandomGenerator (Java 17+)».

«SecureRandom — только для криптографии, не для бизнес-логики».

Бонус — упомянуть JMH как стандарт измерения concurrent-производительности.

## 03 Баг #2 — parallelStream на общем ForkJoinPool

Технический баг про concurrency. Большинство кандидатов видят `.parallel()` и думают «ну, многопоточка — это плохо тут». Senior должен идти глубже: какой пул, какой эффект на остальное приложение, как фиксировать правильно.

### ► Где баг

```
return Arrays.stream(basePrices)
    .parallel()           // <-- использует общий ForkJoinPool.commonPool()
    .filter(...)
    ...;
```

### ► Что не так

#### ■ Почему `.parallel()` — проблема?

→ `parallel()` и `parallelStream()` под капотом используют `ForkJoinPool.commonPool()` — это один общий пул потоков на ВСЁ JVM-приложение. Размер по умолчанию = `Runtime.availableProcessors()` - 1 (на 8-ядерной машине это 7 потоков). Все `parallelStream`-ы во всех сервисах используют этот один пул. Если наш сервис запустил обработку 50 000 SKU с тяжёлой операцией внутри — он на минуты блокирует общий пул, и другие параллельные операции в других сервисах встают. Это классический случай `global state` в Java.

#### ■ Чем это грозит в проде?

→ Сценарий: микросервис обрабатывает 1000 RPS обычных запросов. Каждый из них где-то использует `parallelStream` (агрегация, фильтр коллекции). Параллельно стартует `background-job` на пересчёт скидок: 50 000 SKU × тяжёлая операция / 7 потоков = несколько минут. На эти минуты `common pool` целиком занят `background-задачей`. Все обычные `parallelStream` становятся эффективно `sequential` — задачи стоят в очереди `commonPool`, ждут освобождения потоков. p99 latency API в десятки раз вырастает. И никаких алертов: технически ничего не упало.

#### ■ Как исправить?

→ Два рабочих варианта. Первый — создать свой `ForkJoinPool` под конкретную задачу и передать параллельный стрим внутрь его `submit`. Вторым (Java 21) — использовать `newVirtualThreadPerTaskExecutor`: виртуальные потоки решают проблему «пул заканчивается», их может быть миллион. Для `batch-задач` с `blocking IO` виртуальные потоки — лучший выбор. Для `CPU-bound` — `ForkJoinPool` с разумным числом потоков (обычно число ядер).

### ► Решение 1: свой ForkJoinPool

```
@Service
public class BlackFridayPricingService {

    private final ForkJoinPool pricingPool = new ForkJoinPool(
        Runtime.getRuntime().availableProcessors() // выделенный пул
    );

    public List<BigDecimal> calculate(int[] basePrices) {
        try {
            return pricingPool.submit(() ->
                Arrays.stream(basePrices)
                    .parallel()
            );
        }
    }
}
```

```

        .mapToObj(this::priceFor)
        .collect(Collectors.toList())
    ).get();
} catch (InterruptedException | ExecutionException e) {
    Thread.currentThread().interrupt();
    throw new RuntimeException(e);
}
}

@PreDestroy
void shutdown() {
    pricingPool.shutdown();    // не забыть закрыть!
}
}

```

## ► Решение 2: virtual threads (Java 21+)

```

public List<BigDecimal> calculate(int[] basePrices) {
    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
        List<Future<BigDecimal>> futures = new ArrayList<>(basePrices.length);
        for (int price : basePrices) {
            futures.add(executor.submit(() -> priceFor(price)));
        }
        List<BigDecimal> result = new ArrayList<>(basePrices.length);
        for (Future<BigDecimal> f : futures) {
            result.add(f.get());
        }
        return result;
    } catch (InterruptedException | ExecutionException e) {
        Thread.currentThread().interrupt();
        throw new RuntimeException(e);
    }
}

```

### ▪ Когда что выбирать — свой ForkJoinPool или virtual threads?

→ ForkJoinPool — для CPU-bound задач: математика, парсинг, трансформации без IO. Размер пула = число ядер. Wow-эффекта не даст, но не блокирует common pool. Virtual threads — для IO-bound: блокирующий JDBC, HTTP-вызовы, файловое IO. Их можно создавать миллионы — виртуальный поток ~few KB против ~1 MB у платформенного. Когда виртуальный поток блокируется на IO, он снимается с carrier (платформенного) потока, который тут же берёт другой виртуальный. CPU не простаивает.

#### ФИШКА • Что сказать на собеседовании про parallelStream

.parallel() использует ОБЩИЙ ForkJoinPool.commonPool() на всю JVM.

Один тяжёлый parallelStream может «забить» пул для всего приложения.

Решение: свой ForkJoinPool через submit, либо virtual threads (Java 21+).

Для CPU-bound — выделенный FJP. Для IO-bound — virtual threads.



## 04 Баг #3 — magic filter в стриме

Баг «стилистический», но для Senior он принципиален — это про инженерную культуру, читаемость и поддержку. Если кандидат его не замечает, это знак: код писали без code review.

### ► Где баг

```
.filter(price -> price % 2 != 0)    // <-- что это вообще?
```

### ► Что не так

#### ■ В чём суть проблемы?

→ Это «магическое» условие — фильтр, смысл которого непонятен из кода. «Берём только нечётные цены» — это бизнес-правило или баг? Если правило — должно быть документировано или вынесено в named-predicate. Если баг — почему оно в коде? На code review такой фрагмент должен заставить рецензента остановиться и задать вопрос автору: «А что должно произойти с чётными ценами? Они не попадают в результат — это намеренно?». Если автор не может объяснить — это явный баг.

#### ■ Какие могут быть «легитимные» причины?

→ Их немного, и все маловероятны. (1) Тесты на edge cases — фильтр оставили из отладочного кода. (2) Какой-то странный бизнес-юзкейс типа «скидку даём только на товары с нечётной ценой» — но это нонсенс с точки зрения маркетинга. (3) Способ симулировать ~50% выборку через простое условие — тогда нужно правильное sampling. В реальном коде ВСЕГДА должен быть смысл у каждой строки. Условие без объяснения — это долг для будущих разработчиков.

#### ■ Как правильно?

→ Два варианта в зависимости от ответа на «зачем»: (1) Если фильтр случайный или для отладки — удалить полностью. Если ничего не сломается — фильтр был лишним. (2) Если фильтр действительно нужен бизнесу — вынести в named-метод с понятным именем, добавить Javadoc. Это сразу превращает «магию» в документированный код: filter(this::isEligibleForDiscount). А внутри isEligibleForDiscount — реальная логика с объяснением.

### ► Как должно быть

```
.filter(this::isEligibleForDiscount)

/**
 * Товар eligible для Чёрной пятницы, если он не относится к stop-list:
 * – алкоголь и табак (по закону скидки запрещены)
 * – социальные товары (хлеб, молоко – регулирование цен)
 * – товары собственного производства (нет маржи для скидки)
 */
private boolean isEligibleForDiscount(int price) {
    // конкретная бизнес-логика
    return ...;
}
```

### ► Почему это важно для Senior

#### ■ Что показывает Senior на этом моменте?

→ Способность отделять «работает» от «правильно». Junior смотрит на `filter` и думает: «ну это просто фильтр, всё ок». Middle может заметить, что условие странное, но не настаивает на изменении. Senior останавливается: «это бизнес-правило или баг? Если правило — опишем явно. Если баг — уберём». Это часть code review культуры. Если в коде нельзя понять «зачем» — это код, который через год никто не поймёт. И через два года его трогать бояться, начинается legacy.

#### **ФИШКА** · Правило для code review

На любом code review задавай себе два вопроса: «Это работает правильно?» и «А ЗАЧЕМ это вообще делается?». Второй важнее.

Если на «зачем» нет очевидного ответа — спрашивай у автора.

Идеальный код объясняет себя сам. Если не объясняет — нужен комментарий или рефакторинг в `named`-метод.

Просто имена методов уже документация: `isEligibleForDiscount`, `isPremiumCustomer`, `hasActiveSubscription`.

## 05 Баг #4 — field injection и не-final зависимости

Архитектурный баг про то, как объявлены зависимости класса. Кажется мелочью, но для Senior это маркер «писали без понимания современного Spring и без оглядки на тесты».

### ► Где баг

```
@Service
public class BlackFridayPricingService {

    private Random random = new Random();    // <-- не final, инициализация в поле
    // нет конструктора, нет @RequiredArgsConstructor
}
```

### ► Что не так

#### ■ В чём проблема — несколько связанных пунктов

→ Несколько связанных проблем. (1) Поле не final — может быть случайно перезаписано, в том числе через reflection. Для thread-safe сервиса (а Spring singleton — он всегда thread-safe должен быть) — это потенциальная проблема. (2) Зависимость на конкретную реализацию Random — нельзя подменить в тестах на mock или фиксированный seed. (3) Нет явного объявления зависимостей через конструктор — все зависимости класса размазаны по полям. Если их 5-10, читателю сложно понять «что вообще нужно сервису». (4) Spring инициализирует поля через рефлексию — нельзя сделать инжектируемое поле final.

#### ■ Почему final-поля и constructor injection — стандарт Senior?

→ Иммутабельность зависимостей — основа thread-safe сервиса. После создания объекта его зависимости НЕ должны меняться. final гарантирует это на уровне компилятора. Конструктор делает зависимости явными — открыл класс, видишь параметры конструктора, знаешь что нужно. Это упрощает понимание и тестирование. В Spring Boot 3 + Lombok минимальный правильный код — это @RequiredArgsConstructor над классом и private final над каждой зависимостью. Лаконично и идиоматично.

### ► Как правильно

```
@Service
@RequiredArgsConstructor    // Lombok создаёт конструктор
public class BlackFridayPricingService {

    private final PromotionRepository promotionRepo;    // зависимости явно
    private final RandomGenerator rng;                // интерфейс Java 17+

    // Никакого Random как поле — используем интерфейс,
    // конкретную реализацию подсовываем через @Bean

    public List<BigDecimal> calculate(...) {
        // в коде:
        double r = rng.nextDouble();
    }
}

// В конфигурации Spring:
```

```
@Configuration
class PricingConfig {

    @Bean
    RandomGenerator rng() {
        // в проде: ThreadLocalRandom — нет contention
        return ThreadLocalRandom.current();
    }
}
```

#### ▪ Зачем RandomGenerator вместо Random?

- RandomGenerator — общий интерфейс для всех ГСЧ в Java 17+. Это часть JEP 356: Enhanced Pseudo-Random Number Generators. Реализации: ThreadLocalRandom, SplittableRandom, различные алгоритмы (Xoshiro, L128X, L64X). Через интерфейс можно: (1) Подменять реализации без изменения кода (factory). (2) Mock-ать в тестах через простую заглушку. (3) Использовать улучшенные алгоритмы (Xoshiro быстрее старого Random). Программирование на интерфейсы — базовый принцип. На сервис-уровне зависимость должна быть на RandomGenerator, не на конкретный Random.

### ► Главное преимущество — тесты

```
// Юнит-тест с фиксированными значениями rng
@Test
void calculatesDiscount_givenFixedRandom() {
    RandomGenerator fixedRng = mock(RandomGenerator.class);
    when(fixedRng.nextDouble()).thenReturn(0.2); // всегда 20% скидка
    var promoRepo = mock(PromotionRepository.class);
    var service = new BlackFridayPricingService(promoRepo, fixedRng);

    // Теперь поведение детерминированное — тест воспроизводим
    var result = service.calculate(...);
    assertThat(result).containsExactly(...);
}
```

#### ▪ Главный плюс constructor injection в тестах?

- Не нужен Spring-контекст для юнит-теста. Просто new MyService(mock1, mock2) — и можно тестировать. Контекст Spring поднимается секунды-десятки секунд, на тысячах юнит-тестов это разница в часы CI. Поэтому правило: для бизнес-логики — юнит-тесты с new и mock-ами. Для интераций — @SpringBootTest. С field injection (@Autowired над полем) приходится городить ReflectionTestUtils или поднимать @SpringBootTest даже для простой логики — это технический долг.

#### ФИШКА • Что показать на code review

Не критиковать стиль сам по себе — связать с практическим последствием.

Field injection → проблемы с тестами (привести пример).

Не-final → потенциальная mutability в singleton (опасно).

Конкретная реализация в поле → невозможно подменить (мок, A/B тест, миграция).

## Ещё 4 бага в Java Jub Pro

Выше — четыре технических бага, которые ловит внимательный Middle. Ниже — четыре, на которых отсеивают именно Senior-кандидатов. Среди них тот самый главный коварный — легенда этой задачи, ради которой собес X5 её и даёт. Каждый из четырёх стоял за реальным production-инцидентом. Полный разбор каждого с примерами правильного кода — в Java Jub Pro.

### ► Баг #5 — ГЛАВНЫЙ КОВАРНЫЙ

Самый дорогой баг в коде. Не про многопоточность, не про производительность. Это ОДНА СТРОКА, которая компилируется, проходит все unit-тесты, отработывает локально, не падает в нагрузочном тестировании. И через неделю после релиза становится причиной production-инцидента в реальном крупном российском ритейлере.

```
return new BigDecimal(price * (1 - discount)); // <-- ОДНА строка
```

Что произошло. Маркетинг загрузил скидки на Чёрную пятницу, сервис пересчитал 50 000 цен ночью. В БД сохранились значения вида 245.990000000000000028. На ценнике показалось «245.99 ₽». На кассе пробилось ровно из БД. Банк-эквайер не понял дробных копеек, отклонил авторизацию. Касса встала. Очередь. И так в тысячах магазинов одновременно.

На этом баге отсеивают **80% Senior-кандидатов** в финтех и ритейл. Не потому что он сложный — он очевидный, когда знаешь. Сложно его НАЙТИ, когда не знаешь, потому что код «правильно использует BigDecimal» — что может быть не так?

В Pro: три проблемы внутри ОДНОЙ строки, что именно делает JVM с числом, два рабочих способа фикса, почему один лучше другого, как хранить деньги в Postgres правильно, почему BigDecimal.equals — это тоже баг, чек-лист на code review для денег.

### ► Баг #6 — блокирующий вызов внутри parallelStream

```
.mapToObj(price -> {
    try {
        Thread.sleep(50);
        // ...
    }
})
```

Что здесь делает Thread.sleep? Раньше в коде так писали «защиту от перегрузки внешнего сервиса». Сейчас это антипаттерн, и причина — не сама sleep. На 50 000 SKU с 7 потоками commonPool суммарно получается несколько минут блокировки общего пула. Для production-кода есть нормальный инструмент с распределённым режимом — три буквы.

В Pro: какой именно инструмент, как настроить distributed rate limiter, чем sleep опасен на K8s-нодах с несколькими репликами, и как этот же баг выглядит в Spring Boot-сервисе с интеграцией к downstream API.

### ► Баг #7 — проглоченный InterruptedException

```
catch (InterruptedException e) {
    // ignore <-- классический антипаттерн
}
```

Самый цитируемый антипаттерн многопоточного Java-кода. Описан в первой главе книги Brian Goetz «Java Concurrency in Practice» (JCiP) — настольной для Senior. Проблема не в том, что мы «не обработали исключение». Проблема глубже — мы потеряли сигнал, который JVM передаёт между потоками. И этот сигнал нужен на каждом K8s rolling deployment.

В Pro: что именно ломается при SIGTERM → shutdownNow() → interrupt → проглатывание, три рабочих стратегии обработки InterruptedException по Goetz, почему пустой catch (InterruptedException e) — это автоматический маркер «не читал JCiP», и почему даже компилятор Java не помогает поймать эту ошибку.

## ► Баг #8 — Random для финансовых расчётов

```
double discount = random.nextDouble() * 0.5;
```

Этот баг — НЕ про contention (хотя contention тоже есть, это баг #1 выше). Он про бизнес-логику. Подсказка: представь, что система пересчитывает цену одного и того же товара дважды. Что должно произойти? В коде происходит другое.

В Pro: почему рандомные скидки — баг бизнес-логики, а не баг кода; три способа делать детерминированный расчёт цен (БД-правила, rules engine, hash-based для A/B-тестов); как этот баг отличается от технического contention из главы 02 выше, и почему в финтехе и ритейле детекторы CodeQL ловят такие паттерны до прода.

## ► Что отличает 4 «бесплатных» бага от 4 «Pro»

Bug #1-#4 — те, на которых отсеивают Middle: технические баги (contention на Random, parallelStream на commonPool) и архитектурные стандарты (field injection, magic conditions). Их находит внимательный кандидат с опытом 2-3 года.

Bug #5-#8 — то, что отличает Middle+ от Senior. ★ Главный коварный (BigDecimal) — это вообще не про код, а про понимание IEEE 754 и production-инциденты. Thread.sleep — про rate limiting в распределённой системе. InterruptedException — про graceful shutdown и Goetz JCIIP. Random недетерминизм — про бизнес-логику, не про код.

Все 4 — это разные категории интриги. Если в твоей работе появляется хотя бы одна из них (банк, биржа, ритейл, биллинг, любой high-load с финансами), ты столкнёшься с каждой. Лучше разобраться один раз, чем чинить в проде.

### **JAVA JUB PRO** · Что входит в полный разбор

Подробный разбор всех 8 багов (29 страниц вместо 14 здесь).

★ Развёрнутый разбор главного коварного — историей кейса в крупном ритейлере.

Финальный production-ready код всего сервиса со всеми исправлениями.

Юнит-тесты с edge cases (null, пустой вход, awkward значения цен  $99.99 \times 33.33\%$ ).

Сравнительная таблица «было / стало» по 12 аспектам.

Чек-лист на 30 пунктов — маркеры антипаттернов для code review.



# Финальный production-ready код + чек-лист

В полной версии разбора в Java Jub Pro — собранный production-ready BlackFridayPricingService со всеми исправлениями. Тот код, который мог бы пройти в проде через code review в X5 Tech.



## Что внутри финального кода (фрагмент)

```
/**
 * Сервис расчёта скидочных цен для Чёрной пятницы.
 *
 * Правила расчёта:
 * 1) Скидки берутся из таблицы promotions — детерминированно по SKU и дате.
 * 2) Цены хранятся и считаются в копейках как long для производительности и точности.
 * 3) BigDecimal используется только при возврате результата клиенту.
 * 4) Параллельное вычисление — через выделенный ForkJoinPool, чтобы не блокировать
 *    общий commonPool приложения.
 *
 * Thread safety: класс stateless, все зависимости immutable, безопасен в Spring singleton.
 */
@Service
@RequiredArgsConstructor
public class BlackFridayPricingService {

    private static final int DISCOUNT_SCALE = 2;
    private static final RoundingMode ROUNDING = RoundingMode.HALF_EVEN;

    private final PromotionRepository promotionRepo;
    private final ForkJoinPool pricingPool;

    public List<BigDecimal> calculate(List<Long> skuIds, LocalDate date) {
        // ... 50+ строк с правильной обработкой ошибок,
        //      idempotency, audit-логом, метриками
        //
        //      → продолжение в Java Jub Pro
    }
}
```

В полном файле — все 100+ строк с:

- Транзакционным контекстом и тайм-аутами.
- Обработкой InterruptedException по Goetz (восстановление флага + кастомное исключение).
- Выделенным ForkJoinPool с управлением жизненным циклом через @PreDestroy.
- Поддерживаемыми классами: Promotion (record), PricingException, конфигурация Spring.
- Юнит-тестами с моками PromotionRepository и фиксированными RandomGenerator.
- Тестами edge cases: null-входы, пустые списки, awkward значения цен (99.99 ₺ × 33.33%).



## ► Чек-лист на 30 пунктов — маркеры антипаттернов для code review

В Pro прилагается чек-лист на 30 пунктов. Это не «рекомендации» в общем виде — это конкретные фрагменты кода, которые ДОЛЖНЫ автоматически останавливать тебя на code review. Запомни их, и ты будешь ловить 80% типовых багов с первого прохода.

### ► Фрагмент чек-листа

| Маркер в коде                                              | Что не так                                               |
|------------------------------------------------------------|----------------------------------------------------------|
| <code>new BigDecimal(double)</code>                        | ★ Production-инцидент в финансах → полный разбор в Pro   |
| <code>catch (InterruptedException) { /* ignore */ }</code> | Глава 1 из Goetz JCiP → разбор в Pro                     |
| <code>Thread.sleep(...)</code> для rate limit              | Блокирует поток → нужен library, разбор в Pro            |
| <code>private Random random = new Random()</code>          | Contention под parallel → <code>ThreadLocalRandom</code> |
| <code>parallelStream()</code> в business-логике            | Общий <code>commonPool</code> → свой FJP                 |
| <code>catch (Exception e)</code> — слишком широко          | Конкретные exception типы                                |
| <code>e.printStackTrace()</code> в проде                   | Не логирование → <code>log.error</code> с MDC            |
| REST-вызов внутри <code>@Transactional</code>              | Connection pool leak → Outbox                            |

В Pro — все 30 пунктов плюс пояснение по каждому: ПОЧЕМУ это маркер, КАК фиксить, примеры из реальных PR. Это материал, который превращает разработчика, читающего код, в разработчика, ВИДЯЩЕГО проблемы в коде.

# // Что ещё в Java Jub Pro

Этот разбор — часть большой серии «Собес в X5 Tech / Senior Java». Если этот документ зашёл — там ещё много материала, который не помещается в бесплатный канал.

## ► Бесплатный квиз для самопроверки

Перед тем, как идти в Pro — пройди квиз по X5 Tech Senior Java. 15 вопросов с ловушками, где «очевидный» ответ часто неправильный. Это покажет твой реальный уровень — не по самооценке, а по конкретным ошибкам.

Например, что вернёт `new BigDecimal(0.1).toString()` — «0.1» или «0.10000000000000000555...»? Если ответ «0.1» — посмотри ещё раз, это и есть тот самый главный коварный баг.

## ► Кому Pro полезен в первую очередь

- Идёшь на Senior-собес через 1-3 месяца в банк / биржу / ритейл / финтех.
- В стеке есть деньги — BigDecimal, балансы, лимиты, ордера, цены.
- Хочешь не «учить ответы», а понимать почему Goetz JCIp до сих пор актуальна.
- Делаешь code review в команде и хочешь делать его на уровень выше.
- Готовишь Middle-кандидатов к Senior — нужен материал для менторства.

### JAVA JUB PRO • Что внутри подписки

Все гайды по вакансиям (X5, Совкомбанк, Т1, новые добавляются).

Полные разборы задач дня — каждая с production-кейсом и финальным кодом.

Quiz-сборники с 50+ вопросами под каждую вакансию, с разборами.

Доступ к каналному чату — обсуждение собесов, фидбэки, договорённости.

Стоимость: меньше одного обеда в месяц.

Перейти в Java Jub Pro — @Java\_Jub\_Pro

Если этот разбор был полезен — поделись им с коллегой, который тоже готовится. Хорошие материалы расходятся через людей, не через рекламу.